

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №6

«Шаблоны проектирования и модульное тестирование в Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Подругин Г.Р.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Реализовать порождающий шаблон Factory Method для сервиса уведомлений и проверить его с помощью pytest, pytest-bdd и Mock.

Выбран допустимый минимальный вариант задания с одним шаблоном проектирования. Предметная область — создание уведомлений двух видов, Email и SMS. Обязательные способы проверки: TDD-фреймворк, BDD-фреймворк и Mock-объекты.

Описание решения

Notification — абстрактный продукт, EmailNotification и SmsNotification — конкретные продукты. NotificationService — создатель с фабричным методом create_notification и общей операцией notify. EmailService и SmsService переопределяют фабричный метод. Общая операция работает через интерфейс продукта и записывает результат в переданный поток.

Передача потока в конструктор отделяет бизнес-логику от консоли. StringIO используется для проверки текста, Mock — для проверки единственной записи, отсутствия записи при ошибке и распространения OSError. Шесть тестовых случаев pytest проверяют фабрики и поведение сервиса. Три BDD-сценария описывают Email, SMS и пустое сообщение.

Порядок запуска

Python 3.10 или новее. Из корня репозитория создать виртуальное окружение и установить зависимости командой python -m pip install -r requirements.txt.

```
python lab_06/main.py
python -m pytest lab_06 -v
```

Результаты выполнения

```
$ python lab_06/main.py
EMAIL | Кому: student@example.com | Лабораторная работа готова
SMS | Кому: +70000000000 | Лабораторная работа готова

Итог тестового запуска (полный журнал tests.txt)
$ python -m pytest lab_06 -v --tb=short
============================== 9 passed in 0.02s ===============================
```

Рисунок 1 — Вывод фактического запуска программы

Экранная форма сформирована из сохранённого stdout. Полные журналы: results/demo.txt и results/tests.txt.

Проверка и выводы

Для этой работы успешно выполнено проверок: 9. Общий набор всех шести работ содержит 61 тест; полный прогон завершился без ошибок. Версии среды и журналы сохранены в validation/ и results/tests.txt.

Factory Method позволяет добавлять новый вид уведомлений новым классом создателя и продукта. Девять проверок прошли успешно. Программа выводит локальное представление уведомления и не отправляет реальные письма или SMS.

Источник задания

https://github.com/ugapanyuk/BKIT_2021/wiki/lab_python_test

Текст программы

```
# Файл notifications.py

"""Порождающий паттерн Factory Method в сервисе уведомлений."""
from abc import ABC, abstractmethod
from typing import TextIO


class Notification(ABC):
    @abstractmethod
    def render(self, recipient: str, message: str) -> str:
        raise NotImplementedError


class EmailNotification(Notification):
    def render(self, recipient, message):
        return f"EMAIL | Кому: {recipient} | {message}"


class SmsNotification(Notification):
    def render(self, recipient, message):
        return f"SMS | Кому: {recipient} | {message}"


class NotificationService(ABC):
    def __init__(self, output: TextIO):
        self.output = output

    @abstractmethod
    def create_notification(self) -> Notification:
        raise NotImplementedError

    def notify(self, recipient: str, message: str) -> str:
        if not recipient.strip() or not message.strip():
            raise ValueError("Получатель и сообщение должны быть непустыми")
        rendered = self.create_notification().render(recipient, message)
        self.output.write(rendered + "\n")
        return rendered


class EmailService(NotificationService):
    def create_notification(self):
        return EmailNotification()


class SmsService(NotificationService):
    def create_notification(self):
        return SmsNotification()

# Файл main.py

"""Локальная демонстрация: сообщения выводятся в консоль."""
```

Текст программы Продолжение

```
import sys
from notifications import EmailService, SmsService


if __name__ == "__main__":
    for service, recipient in ((EmailService(sys.stdout), "student@example.com"),
                               (SmsService(sys.stdout), "+70000000000")):
        service.notify(recipient, "Лабораторная работа готова")

# Файл test_factory.py

from io import StringIO
from unittest.mock import Mock
import pytest
from notifications import EmailService, SmsService, EmailNotification,
    SmsNotification


@pytest.mark.parametrize("service, product", [
    (EmailService, EmailNotification), (SmsService, SmsNotification)])
def test_factory_returns_correct_product(service, product):
    assert isinstance(service(StringIO()).create_notification(), product)


def test_email_format():
    output = StringIO()
    result = EmailService(output).notify("student@example.com", "Готово")
    assert result == "EMAIL | Кому: student@example.com | Готово"
    assert output.getvalue() == result + "\n"


def test_mock_output_called_once():
    output = Mock()
    result = SmsService(output).notify("+70000000000", "Готово")
    output.write.assert_called_once_with(result + "\n")


def test_empty_message_does_not_write():
    output = Mock()
    with pytest.raises(ValueError):
        SmsService(output).notify("+70000000000", " ")
    output.write.assert_not_called()


def test_output_failure_propagates():
    output = Mock()
    output.write.side_effect = OSError("disk full")
    with pytest.raises(OSError, match="disk full"):
        EmailService(output).notify("student@example.com", "Готово")

# Файл features/notifications.feature
```

Текст программы Продолжение

```
Feature: Notification factory method
  Scenario: Email notification
    Given an email service
    When I notify the student
    Then the notification starts with EMAIL

  Scenario: SMS notification
    Given an sms service
    When I notify the student
    Then the notification starts with SMS

  Scenario: Empty notification is rejected
    Given an email service
    When I submit an empty message
    Then no notification is written
```

```
# Файл test_bdd.py

from io import StringIO
import pytest
from pytest_bdd import given, when, then, scenarios, parsers
from notifications import EmailService, SmsService

scenarios("features/notifications.feature")


@given(parsers.parse("an {channel} service"), target_fixture="service")
def service(channel):
    return {"email": EmailService, "sms": SmsService}[channel](StringIO())


@when("I notify the student", target_fixture="message")
def notify(service):
    return service.notify("student", "Готово")


@then(parsers.parse("the notification starts with {prefix}"))
def check_prefix(message, prefix):
    assert message.startswith(prefix + " |")


@when("I submit an empty message")
def invalid_message(service):
    with pytest.raises(ValueError):
        service.notify("student", "")


@then("no notification is written")
def no_output(service):
    assert service.output.getvalue() == ""
```